

Team Execution Checklist

Oil Spill Detection & AIS Attribution — 3-Day Build

Sequential, per-person checklists. Each person owns one domain end-to-end on their own machine — no model or processing step is split across two people. Check items off in order within your track. Sync points with other people are called out explicitly so you know when to stop and integrate rather than push ahead solo.

Refer to `PRD_Oil_Spill_AIS_Attribution.md` for full data contracts (§7) any time a task below says "per contract" — note that some output filenames below are simplified/renamed from the PRD's schema names for clarity; the underlying structure is the same.

P1 — Satellite Imagery, Segmentation & CFAR (Vision)

Day 1

- [] Set up Google Earth Engine account/auth, confirm access works
- [] Pick the demo event: fix one real or realistic scenario (location, date, time window) — communicate this to the whole team immediately, everyone else builds against it
- [] Write GEE query to pull Sentinel-1 GRD scene(s) for the chosen AOI/date
- [] Apply calibration + speckle filtering (Refined Lee) + terrain correction
- [] Get the Krestenitis et al. labelled SAR oil-spill dataset (oil spill / look-alike / ship / land / sea classes)
- [] Set up U-Net or DeepLabv3+ with a pretrained backbone, fine-tune on the dataset (time-box this — a reasonable model by end of Day 1 is the target, not a perfect one)
- [] Run segmentation on the calibrated demo scene; vectorize mask to polygon(s) using `rasterio` / `shapely`
- [] **Sync checkpoint (end of Day 1):** slick polygon output matches PRD §7.1 geometry format; hand off to P3 for the look-alike filter pass and to P2 for particle seeding

Day 2

- [] Run a classical CFAR (Constant False Alarm Rate) ship detector across the raw SAR raster, independent of AIS — extract physical metal-ship coordinates
- [] Package slick polygon + CFAR ship coordinate list into a single output (`sar_detection_output.json`)
- [] Hand off CFAR ship coordinates to P5 for AIS cross-referencing (dark-vessel check)
- [] **Sync checkpoint (end of Day 2):** confirm P5 has everything needed to classify AIS-matched / dark / AIS-only

Day 3

- [] Verify and cache one clear spill scene and one look-alike scene locally (the look-alike scene is P3's live-rejection demo — make sure it actually trips the wind gate)
 - [] Support integration testing: confirm your two outputs (slick polygon, CFAR coordinates) are being consumed correctly downstream
 - [] Support final rehearsal — be ready to explain the segmentation and CFAR steps if a judge asks
-

P2 — Ocean Physics & Backtracking Engine

Day 1

- [] Install OpenDrift/OpenOil; confirm a basic forward-run example works before touching real data
- [] Download and slice local NetCDF files for HYCOM currents and ERA5 winds, scoped to the demo AOI and time window
- [] Get familiar with OpenDrift's backward-integration (negative timestep) mechanics on a toy example
- [] **Sync checkpoint (end of Day 1):** confirm with P1 that you'll receive a slick polygon first thing Day 2 — don't wait idle, use any remaining time to pre-build the particle-seeding function against a dummy polygon

Day 2

- [] Take the slick polygon from P1, seed particles uniformly across the area (or at H3 cell centroids once P4's utility is available — coordinate timing with P4)
- [] Run backward Lagrangian drift integration (–15 min internal steps, checkpointed at –6h, –12h, –24h) with a ~3% wind-drift factor and stochastic diffusion
- [] Export the raw particle cloud as `(lat, lon, time)` tuples into `drift_particles.json`
- [] **Sync checkpoint (end of Day 2):** hand off raw particle cloud to P4 for H3 re-binning; confirm P4 can parse your output format without modification

Day 3

- [] Pre-compute and cache two ocean-forcing variants (different wind/current inputs) so you can rerun live if a judge asks a "what if" question during Q&A
- [] Support integration testing on the corridor output (with P4) to confirm the physics-to-hex handoff is correct
- [] Support final rehearsal — be ready to explain why backward drift produces a spread region, not a point (this will get asked)

P3 — Look-Alike Filter & Scoring Engine

Day 1

- [] Set up ERA5 wind data access (Copernicus Climate Data Store)
- [] Implement the wind-speed gate: reject candidate regions with wind < 2 m/s or > 12 m/s
- [] Implement damping ratio calculation (mean backscatter inside dark region vs. local sea background)
- [] Implement shape features: eccentricity, solidity, perimeter²/area complexity
- [] Implement alignment check: slick major axis vs. plausible vessel heading/shipping lane
- [] Once P1 hands off the polygon, run it through the full filter — confirm you can produce both a "confirmed" and a "rejected" example. **You need a working rejection example for the demo**, so deliberately test on the low-wind look-alike scene P1 is caching
- [] Output must match PRD §7.1 `lookalike_filter` block exactly

Day 2

- [] Design the scoring feature set: corridor overlap, heading alignment, speed anomaly (pumping-speed detection), AIS-gap history
- [] Build the interpretable scoring model (logistic regression or gradient boosting — explicitly not a deep net)
- [] Define and document explicit weights for each feature now — write them down, don't leave them implicit in code, since they go directly into the case file later
- [] Build and test the scoring logic against **mock/placeholder candidate data** — don't wait on P4's real output to start this, or you lose a full day
- [] **Sync checkpoint (end of Day 2):** confirm with P4 what the real candidate vessel object will look like (per PRD §7.3), so Day 3 is a swap-in, not a rebuild

Day 3

- [] Receive real matched candidates from P4, swap in for your mock data, calculate feature scores
- [] Finalize scoring weights after testing against the full demo scenario

- [] Confirm the "no candidate above threshold" null-result logic triggers correctly when appropriate
 - [] Output `ranked_suspects.json` matching PRD §7.4
 - [] Help construct a second, minimal test scenario that deliberately produces a null result, for the "restraint" demo beat
 - [] Support final rehearsal
-

P4 — H3 Core Indexing & Spatiotemporal Matching

Day 1

- [] Set up `h3-py` at Resolution 7 — write this into a shared constants file the whole team imports; do not let this get hardcoded differently in different modules
- [] Build core conversion utilities: `polygon_to_hex`, `point_to_hex`, `k_ring_expansion`
- [] Share these utilities with P2 (for particle-cloud re-binning) and P5 (for AIS hex-indexing) as soon as they're stable — this is the piece everyone else depends on, prioritize a working version early over a perfect one

Day 2

- [] Take P2's raw particle trajectories and convert them into the time-indexed H3 corridor: `{t0: H_0, t_minus_6h: H_-6, t_minus_12h: H_-12, t_minus_24h: H_-24}`, with particle-density counts per cell
- [] Perform fast hash set-intersection between the backtracked corridor and P5's standardized AIS hex index
- [] **Sync checkpoint (end of Day 2):** confirm corridor object matches PRD §7.2 exactly, and that P3 knows the shape of the candidate object you'll be sending

Day 3

- [] Implement bounded k-ring expansion ($k=1, k=2$) with confidence-decay penalties ($1/(1+k)$) as a fallback when primary matches are sparse
 - [] Output `candidate_vessels.json` matching PRD §7.3 exactly, hand off to P3
 - [] Stress-test the matching engine against the full demo scenario — confirm timing is fast enough for a live demo (this should be near-instant; if it isn't, the lookup logic has a bug, not the physics)
 - [] Support final rehearsal, and be ready to debug schema mismatches on either side of your stage (P2 → P4 and P4 → P3) since you sit in the middle of the pipeline
-

P5 — AIS Ingestion, Dark Vessel Classifier & Case File Export

Day 1

- [] Decide AIS source for the demo scenario: real free-coverage data (MarineCadastre.gov / Global Fishing Watch) if the demo region has it, otherwise synthetic AIS — confirm this choice with P1 based on the chosen demo event/location
- [] Pull or generate AIS data for the region and time window
- [] Standardize: snap all positions to 5-minute intervals
- [] Convert all AIS coordinates to H3 hex IDs using P4's shared utility script as soon as it's available
- [] **Sync checkpoint (end of Day 1):** confirm your AIS index format matches what P4's matching engine expects — this is a two-way contract, talk to P4 directly rather than guessing the format

Day 2

- [] Once P1 hands off CFAR ship coordinates, cross-reference every CFAR detection against active AIS records
- [] Classify each detection: AIS-matched / dark vessel (SAR-only, no broadcast) / AIS-only — per PRD §Stage 4 Part B table

- [] Produce the dark-vessel candidate list, flagged separately from the general suspect pool
- [] **Sync checkpoint (end of Day 2):** confirm at least one dark-vessel case exists in your test data — if the demo scenario doesn't naturally produce one, deliberately construct a synthetic test case that does, since this is a required demo beat

Day 3

- [] Build the automated case file exporter (JSON/PDF, e.g. via ReportLab) bundling scene ID, processing parameters, drift config, corridor, AIS query bounds, ranked suspects, dark vessels, and a SHA-256 input-data hash — per PRD §7.5
- [] Confirm the exported case file is downloadable directly from the UI (coordinate with P6 on where the download button lives)
- [] Double-check the dark-vessel flag is visually distinct in the final UI, not buried in the general list (coordinate with P6)
- [] Support final rehearsal, specifically the dark-vessel and case-file-export demo beats

P6 — Full-Stack UI, Map Dashboard & Demo Reliability

Day 1

- [] Set up the frontend project (React + MapLibre/Deck.gl, or Streamlit if time is tighter — decide early and don't switch later)
- [] Set up repo structure, `.gitignore`, and `.env` handling covering all credential/secret files before anyone commits code
- [] Confirm with each person pulling external data (P1, P2, P3, P5) that they're using environment variables for API keys, not hardcoding them
- [] Build the base map layer (satellite/street base, AOI centered on the chosen demo scenario)
- [] Stub out the layer structure (SAR raster, slick polygon, hex corridor, AIS tracks) even before real data exists, so other people can plug in as they finish

Day 2

- [] Build the **time slider** UI component, driving through observation time → -6h → -12h → -24h
- [] Wire the time slider to animate the H3 hex corridor (color/opacity by particle density) once P4 has real corridor data
- [] Add the AIS vessel track overlay layer, wired to P5's data once available
- [] Add the SAR raster overlay layer, wired to P1's processed scene

Day 3

- [] Build the Suspect Ranking Table (with per-feature score bars), wired to P3's `ranked_suspects.json`
- [] Build the Dark Vessel Alert Banner, wired to P5's dark-vessel output — visually distinct (color/icon/pulse) from regular ranked suspects, not buried in the same list
- [] Implement the explicit "**no candidate identified**" **null-result state** in the UI — this must be a real, demoable state, not just a fallback message
- [] Add the case-file download button, wired to P5's export function
- [] **Your most important task:** pre-cache every single piece of data needed for the demo scenario locally (SAR scene, currents, wind, AIS, corridor, scores) so the live demo has zero dependency on internet/API calls
- [] Run a full offline dry-run of the demo yourself before the team rehearsal
- [] Lead the final rehearsal: time it, confirm every required demo beat fires correctly (look-alike rejection, corridor animation, dark-vessel flag, ranked list with breakdown, null-result case, case-file export)

Clean Data Flow Between the Six Machines





No model, dataset, or processing step should ever need to be edited on two different people's machines. If you find yourself needing something from another domain (e.g. needing to tweak segmentation output while working on scoring), that's a signal to talk to the owner and get a fixed output handed to you – not to start editing their code yourself.

Cross-Team Sync Points (all six, mandatory)

- [] **End of Day 1:** P1 has handed off a slick polygon to P3 and P2; P4's H3 utilities are shared and usable by P2/P5; P5's AIS hex index format is confirmed against P4. Do not start Day 2 work until these three handoffs are confirmed – mismatched formats compound fast.
- [] **End of Day 2:** full corridor → candidates chain runs end-to-end at least once (P2 → P4 → P5/P3), even with rough/placeholder UI. This is the point where "does the core idea actually work" gets answered.
- [] **Day 3, mid-day:** full pipeline + UI integrated, all demo beats present even if rough.
- [] **Day 3, evening:** full rehearsal against cached data only, no live API calls, timed.

Keep this file updated as tasks complete – strike through or check off items so the whole team can see progress at a glance.